Deployment Guide

Revision 2026.10.25 | 25 October 2026 | Howick-Pakuranga Chess Club

Project boundary and release state

Use GitHub hpchessclub/hpcc, Vercel team hpcc2 and HPCC's existing Supabase project. This is a separate project from Auckland Knights. Do not copy AKCC bank accounts, contacts, secrets or records. The existing preview/production address is https://hpcc-flax.vercel.app/. Review domain routing in the current HPCC Vercel project before changing the club's public domain. Code deployment does not apply SQL, choose bank defaults, verify email configuration or enable reminders.

Inspect and preserve the database

Back up the HPCC database and inspect its current schema, constraints, triggers, RLS and grants before migration. No connected database credential was used to apply this release to production. The migrations were validated against isolated fixtures based on HPCC schema definitions; this is not confirmation of production migration status. Never execute hpcc_rebuild.sql as a routine upgrade: it resets application data. Preserve Auth users, profile IDs, membership IDs, receipts and Storage objects.

Additive migration order

Review the existing HPCC membership numbering/renewal upgrades first. Approval uses the existing assign_membership_id function and membership option validity fields. The older v4_7 manual tournament approval function may remain installed, but the new approval route uses v4_8. Apply supabase/migration_v4_8_bank_policies_reminders.sql to HPCC only. It adds separate default bank flags, optional tournament policy text, payment method/reference/instruction/consent/reminder fields, an RLS-protected reminder log and two service-role-only transaction functions. It creates the private audit table only if absent and restricts client access. The script is repeatable and contains no reset, production records or actual account numbers. If unique default indexes find ambiguous existing defaults, resolve the configuration explicitly before continuing. Verify schema cache refresh after commit. The bank claim function serialises matching registration identities and returns an identical existing pending transfer/reference. The approval function locks the row and atomically writes one receipt, status update and approver audit; repeat approval is idempotent. Membership activation reuses the existing ID allocator. Test role denial, cancellation/refund rejection, transaction rollback and concurrency in an isolated project before authorising production payment approval.

Required HPCC environment configuration

Set NEXT_PUBLIC_SITE_URL to the correct HTTPS HPCC address. Set NEXT_PUBLIC_SUPABASE_URL and the public Supabase key variables required by the existing server/browser helpers. SUPABASE_SERVICE_ROLE_KEY is private and is also used to sign idle-session and resume tokens. STRIPE_SECRET_KEY and STRIPE_WEBHOOK_SECRET must belong to HPCC. Never commit their values. Configure HPCC's EMAIL_PROVIDER, sender identity and either SMTP credentials or RESEND_API_KEY. Inspect .env.example for the existing supported variables. NZ Post lookup remains optional and uses HPCC-approved credentials only; manual address entry remains available. Use Supabase Auth Site URL and allowed callback URLs for HPCC; remove production localhost recovery redirects. Existing Auth accounts are retained. Admin sessions now require a signed idle cookie after a fresh sign-in. Profile email/role changes are not exposed by the self-service profile endpoint. Current-password-verified password change clears the session and returns to login.

Bank setup and compatibility

Actual bank details are configured only in the private admin portal/database, never in source or these documents. Account saves independently probe name, account_name, bank_account_name, account_number and bank_account_number and populate whichever aliases exist. Errors reset the Save state. Configure one active membership default and one active tournament default. A specific active tournament account overrides the tournament default. Missing, inactive or ambiguous selections fail closed. Referenced accounts cannot be removed through admin; deactivate them instead. Check names, account number, fee, reference and snapshot in an isolated bank registration.

Stripe and refund setup

Use HPCC webhook signing configuration for checkout completion, delayed-payment success/failure, checkout expiry and charge.refunded. Full refunds are reconciled to refunded/cancelled tournament records while retaining receipts. Partial refunds stay active for manual review. Completion replays cannot restore cancelled entries. Completed or processing payments must never be manually approved. Use Stripe test mode and fake player information for validation. Never refund, delete, charge or send confirmation emails to real records merely to test the release.

Reminder scheduler activation

Set a strong private CRON_SECRET and enable ENABLE_PAYMENT_REMINDERS=true only after approving HPCC email and payment configuration. With the flag absent or false, no reminder email is sent. The scheduled endpoint is /api/cron/payment-reminders and requires Authorization: Bearer with the configured secret. vercel.json schedules 20:00 UTC daily. Each run attempts at most 50 eligible entries, prioritising those least recently reminded and excluding entries already attempted that New Zealand day. For higher volume or shorter closing windows, configure an external authenticated scheduler at a suitable frequency; do not assume the daily batch covers unlimited recipients. Vercel plan limits still apply. Eligibility is checked against current entry state, duplicates, event status, the two-hour quiet period and the earlier of registration close/start. Date-only close dates are interpreted at the end of that New Zealand day. Stripe status is checked immediately before sending; paid, refunded, processing and bank entries are suppressed. New Zealand day and unique registration/day keys prevent repeat attempts. Queue/send/failure states are logged. If a process dies after provider acceptance, review the queued record rather than retrying blindly. Sent means provider acceptance, not delivery or opening. Resume tokens are signed and expiry-limited. The GET displays a confirmation page; checkout continuation occurs on POST. Bank, cancelled, refunded and closed-event entries are excluded. Open eligible Stripe sessions are reused; expired unpaid sessions can be replaced under the same registration after another payment check. Snapshot fees are used, not browser-supplied amounts. This flow does not create a new registration.

Dashboard definitions and access

Admin dashboard data is loaded server-side with pagination and complete-result checks. Only aggregates and public event labels reach the chart component. Views include Overview, Membership, Tournament Registrations, Events, Enquiries and Absences; filters are 7/30/90 days and tournament. Membership/enquiries are never narrowed by tournament. New members come from membership creation, not renewal receipts. Membership and tournament revenues use separate paid ledger receipts; pending transfers are excluded. Active members and awaiting enquiries are current totals. Upcoming events cover the next selected number of days. Absent players are grouped privately by player/event/round. Non-unique event-name matches are flagged rather than guessed.

Build, checks and deployment

Run npm ci, npx tsc --noEmit, the relevant Node tests (including bank-upgrade.test.mjs and hpcc-upgrade.test.cjs) and npm run build. Use fake fixtures and an isolated PostgreSQL test engine for atomicity/concurrency tests. Run authentication expiry, profile allowlist, policy consent and public export privacy tests. Deploy only HPCC changes to the existing Vercel project. Monitor build status and inspect the deployed URLs. Verify /join, tournament pages, login and both PDF download links. Do not claim an authenticated workflow was live-tested if no authorised session was available.

Release verification and rollback

Verify migrations and RLS separately from deployment. Check defaults, a synthetic transfer's immutable instructions, one-receipt approval, consent snapshots, filtered dashboard totals and disabled reminders before enabling real sends. Existing legacy records are preserved without inventing policy or bank information. Public player outputs must use an allowlist without DOB/email/phone/consent/bank/payment identifiers; detailed exports require admin permissions. A code rollback does not roll back SQL or payment records. Keep additive fields, stop the scheduler if necessary and investigate logs without exposing secrets. No bank details or secrets should be added to support documents or GitHub.